

Planificación completa de exposición - Grupo 3

Clase 2: DQL + Práctica aplicada al proyecto

1. Datos generales

Elemento	Detalle
Tema de la clase	DQL y práctica de consultas aplicadas al proyecto
Unidad	Base de Datos / SQL
Modalidad	Teórico-práctica
Duración sugerida	2 horas
Herramienta	Oracle SQL Developer / SQL Plus
Base de datos de práctica	Sistema de ventas
Integrantes	Grupo 3

2. Objetivo de la clase

Explicar y aplicar sentencias DQL mediante consultas SELECT, filtros, ordenamientos, agrupaciones, condiciones con HAVING, INNER JOIN y subconsultas básicas, con el fin de que los estudiantes puedan consultar, analizar y depurar información dentro de una base de datos relacionada con el proyecto.

3. Resultado de aprendizaje

- Comprender qué es DQL y su importancia en SQL.
- Utilizar SELECT, WHERE y ORDER BY para consultar y filtrar datos.
- Aplicar GROUP BY y HAVING para generar resultados agrupados.
- Relacionar tablas mediante INNER JOIN.
- Utilizar subconsultas básicas.
- Identificar y corregir errores comunes en sentencias SQL.
- Aplicar consultas DQL en una base de datos del proyecto.

4. Contenidos de la clase

Bloque	Contenido
Bloque 1	Introducción a DQL
Bloque 2	Sentencias principales de DQL
Bloque 3	Depuración de sentencias SQL
Bloque 4	Receso
Bloque 5	Práctica DQL aplicada al proyecto
Bloque 6	Evaluación formativa
Bloque 7	Cierre y reflexión

5. Distribución del tiempo

Bloque	Actividad	Tiempo
Bloque 1	Introducción a DQL	10 min
Bloque 2	Explicación de sentencias DQL	30 min
Bloque 3	Depuración de sentencias	20 min
Bloque 4	Receso	10-15 min
Bloque 5	Práctica DQL aplicada al proyecto	40 min
Bloque 6	Evaluación formativa	10 min

Bloque 7	Cierre	10 min
----------	--------	--------

6. Desarrollo de la clase

Bloque 1: Introducción

Contenido: qué es DQL, diferencia entre DQL y otros sublenguajes SQL, e importancia de DQL en proyectos reales.

DQL significa Data Query Language o Lenguaje de Consulta de Datos. Su función principal es consultar información almacenada en una base de datos. La sentencia más importante de DQL es SELECT.

En proyectos reales, DQL permite obtener reportes, revisar registros, analizar ventas, consultar clientes, verificar productos y generar información útil para la toma de decisiones.

Pregunta inicial	Respuesta esperada
¿Para qué sirve una consulta SELECT?	Para mostrar información almacenada en una o varias tablas.
¿Qué diferencia existe entre consultar datos y modificar datos?	Consultar solo lee información; modificar altera los registros.
¿Por qué es importante filtrar la información?	Porque permite obtener datos específicos y relevantes.

Bloque 2: DQL

Contenido: SELECT, WHERE, ORDER BY, GROUP BY, HAVING, INNER JOIN y subconsultas básicas.

SELECT

SELECT permite mostrar información de una o varias tablas.

```
SELECT nombres, ciudad
FROM CLIENTE;
```

Explicación: Muestra los nombres y la ciudad de todos los clientes registrados.

WHERE

WHERE permite filtrar registros según una condición.

```
SELECT nombres, ciudad
FROM CLIENTE
WHERE ciudad = 'Ambato';
```

Explicación: Muestra únicamente los clientes que viven en Ambato.

ORDER BY

ORDER BY permite ordenar los resultados de una consulta.

```
SELECT nombre_producto, precio
FROM PRODUCTO
ORDER BY precio DESC;
```

Explicación: Muestra los productos ordenados desde el precio más alto hasta el más bajo.

GROUP BY

GROUP BY permite agrupar registros para aplicar funciones como COUNT, SUM, AVG, MIN o MAX.

```
SELECT estado, COUNT(*) AS cantidad_facturas
FROM FACTURA
GROUP BY estado;
```

Explicación: Cuenta cuántas facturas existen por cada estado.

HAVING

HAVING permite filtrar resultados agrupados. Se usa después de GROUP BY.

```
SELECT id_factura, SUM(cantidad * precio_unitario) AS total
FROM DETALLE_FACTURA
GROUP BY id_factura
HAVING SUM(cantidad * precio_unitario) > 100;
```

Explicación: Muestra únicamente las facturas cuyo total supera los 100 dólares.

INNER JOIN

INNER JOIN permite relacionar dos o más tablas y muestra solo los registros que tienen coincidencia.

```
SELECT f.id_factura, c.nombres AS cliente, f.estado
FROM FACTURA f
INNER JOIN CLIENTE c
ON f.id_cliente = c.id_cliente;
```

Explicación: Muestra las facturas junto con el nombre del cliente correspondiente.

Subconsultas básicas

Una subconsulta es una consulta dentro de otra consulta.

```
SELECT nombre_producto, precio
FROM PRODUCTO
WHERE id_producto IN (
    SELECT id_producto
    FROM DETALLE_FACTURA
);
```

Explicación: Muestra los productos que han sido vendidos al menos una vez.

Bloque 3: Depuración de sentencias

Actividad: se mostrarán consultas con errores para que la clase identifique el problema y proponga la corrección.

Error 1: Texto sin comillas

Consulta con error:

```
SELECT nombres, ciudad
FROM CLIENTE
WHERE ciudad = Ambato;
```

Error: El texto debe ir entre comillas simples.

Corrección:

```
SELECT nombres, ciudad
FROM CLIENTE
WHERE ciudad = 'Ambato';
```

Error 2: INNER JOIN sin ON

Consulta con error:

```
SELECT f.id_factura, c.nombres
FROM FACTURA f
INNER JOIN CLIENTE c;
```

Error: Falta la condición de unión ON.

Corrección:

```
SELECT f.id_factura, c.nombres
FROM FACTURA f
INNER JOIN CLIENTE c
ON f.id_cliente = c.id_cliente;
```

Error 3: Uso incorrecto de SUM en WHERE

Consulta con error:

```
SELECT id_factura, SUM(cantidad * precio_unitario) AS total
FROM DETALLE_FACTURA
WHERE SUM(cantidad * precio_unitario) > 100
GROUP BY id_factura;
```

Error: No se puede usar una función de agregación en WHERE.

Corrección:

```
SELECT id_factura, SUM(cantidad * precio_unitario) AS total
FROM DETALLE_FACTURA
GROUP BY id_factura
HAVING SUM(cantidad * precio_unitario) > 100;
```

Error 4: Falta de GROUP BY

Consulta con error:

```
SELECT estado, COUNT(*) AS cantidad
FROM FACTURA;
```

Error: Cuando se usa una columna normal junto con una función de agregación, se debe usar GROUP BY.

Corrección:

```
SELECT estado, COUNT(*) AS cantidad
FROM FACTURA
GROUP BY estado;
```

Error 5: Subconsulta con varias columnas

Consulta con error:

```
SELECT nombre_producto, precio
FROM PRODUCTO
WHERE id_producto IN (
    SELECT id_producto, cantidad
    FROM DETALLE_FACTURA
);
```

Error: La subconsulta usada con IN debe devolver una sola columna.

Corrección:

```
SELECT nombre_producto, precio
FROM PRODUCTO
WHERE id_producto IN (
    SELECT id_producto
    FROM DETALLE_FACTURA
);
```

Bloque 4: Receso

Duración: 10 a 15 minutos.

Bloque 5: Práctica DQL aplicada al proyecto

Los estudiantes trabajarán sobre una base de datos relacionada con ventas. Primero se crearán las tablas, luego se insertarán registros relacionados y finalmente se ejecutarán consultas DQL.

N.º	Actividad	Sentencia principal
1	Crear la base de datos y tablas del proyecto	CREATE TABLE
2	Insertar registros relacionados	INSERT INTO
3	Consultar todos los clientes	SELECT
4	Mostrar clientes de Ambato	WHERE
5	Mostrar productos ordenados por precio	ORDER BY
6	Mostrar facturas pagadas	WHERE
7	Contar facturas por estado	COUNT + GROUP BY
8	Calcular total por factura	SUM + GROUP BY
9	Mostrar facturas mayores a 100	HAVING
10	Relacionar clientes con facturas	INNER JOIN
11	Mostrar productos vendidos	Subconsulta
12	Detectar y corregir errores	Depuración SQL

Consultas para la práctica

1. Mostrar clientes de Ambato

```
SELECT id_cliente, nombres, ciudad
FROM CLIENTE
WHERE ciudad = 'Ambato';
```

2. Mostrar productos con precio mayor a 20

```
SELECT nombre_producto, precio
FROM PRODUCTO
WHERE precio > 20
ORDER BY precio DESC;
```

3. Mostrar facturas pagadas

```
SELECT id_factura, fecha, estado
FROM FACTURA
WHERE estado = 'PAGADA';
```

4. Contar facturas por estado

```
SELECT estado, COUNT(*) AS cantidad_facturas
FROM FACTURA
GROUP BY estado;
```

5. Calcular total por factura

```
SELECT id_factura,
       SUM(cantidad * precio_unitario) AS total_factura
FROM DETALLE_FACTURA
GROUP BY id_factura;
```

6. Mostrar facturas mayores a 100

```
SELECT id_factura,
       SUM(cantidad * precio_unitario) AS total_factura
FROM DETALLE_FACTURA
GROUP BY id_factura
HAVING SUM(cantidad * precio_unitario) > 100;
```

7. Mostrar facturas con cliente

```
SELECT f.id_factura,  
       c.nombres AS cliente,  
       f.estado  
FROM FACTURA f  
INNER JOIN CLIENTE c  
ON f.id_cliente = c.id_cliente;
```

8. Mostrar productos vendidos

```
SELECT nombre_producto, precio  
FROM PRODUCTO  
WHERE id_producto IN (  
    SELECT id_producto  
    FROM DETALLE_FACTURA  
);
```

Bloque 6: Evaluación formativa

Modalidad: participación oral, resolución de consultas y corrección de errores.

1. ¿Qué es DQL?
2. ¿Qué sentencia principal pertenece a DQL?
3. ¿Para qué sirve WHERE?
4. ¿Qué diferencia hay entre WHERE y HAVING?
5. ¿Para qué sirve GROUP BY?
6. ¿Qué hace INNER JOIN?
7. ¿Qué es una subconsulta?
8. ¿Por qué no se puede usar SUM() directamente en WHERE?
9. ¿Qué error existe si se omite el ON en un JOIN?
10. ¿Por qué es importante validar los datos antes de consultar?

Bloque 7: Cierre

Actividades: socialización de errores encontrados durante la práctica, retroalimentación sobre el uso correcto de SELECT, WHERE, GROUP BY, HAVING, INNER JOIN y subconsultas, y reflexión sobre la importancia de validar datos y relaciones entre tablas.

Pregunta de cierre	Propósito
¿Qué errores fueron más comunes durante la práctica?	Identificar dificultades frecuentes.
¿Por qué es importante validar los datos?	Relacionar calidad de datos con consultas correctas.
¿Por qué son importantes las relaciones entre tablas?	Comprender el valor de las claves y los JOIN.
¿Cómo ayuda DQL en un proyecto real?	Relacionar SQL con reportes y toma de decisiones.
¿Qué consulta fue más difícil de construir?	Promover reflexión sobre el aprendizaje.

Archivos utilizados para la clase

Archivo	Uso
Presentación HTML	Explicación de conceptos y evaluación formativa
Script SQL del proyecto	Creación de tablas, inserción de registros y consultas
Ejercicios prácticos	Participación de los estudiantes
Tabla de errores	Depuración de sentencias
Evidencia de resultados	Registro de consultas ejecutadas y salidas obtenidas

Evidencia de aprendizaje

Los estudiantes deberán presentar una tabla de resultados como la siguiente:

N.º	Consulta ejecutada	Tema aplicado	Resultado obtenido	Corrección realizada
1	Cientes de Ambato	WHERE	Cientes filtrados	No aplica
2	Productos por precio	ORDER BY	Productos ordenados	No aplica
3	Facturas por estado	COUNT + GROUP BY	Cantidad por estado	No aplica
4	Facturas mayores a 100	HAVING	Facturas filtradas	No aplica
5	Facturas con cliente	INNER JOIN	Facturas relacionadas	No aplica
6	Error de comillas	WHERE	Error detectado	Agregar comillas
7	JOIN sin ON	INNER JOIN	Error detectado	Agregar condición ON
8	SUM en WHERE	HAVING	Error detectado	Cambiar WHERE por HAVING

Conclusión de la planificación

Esta clase permite que los estudiantes comprendan DQL no solo desde la teoría, sino desde la práctica. La combinación de explicación, consultas aplicadas, depuración de errores y evaluación formativa ayuda a fortalecer el uso correcto de SQL en proyectos reales.